ECOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

UNIVERSITÉ HASSAN II DE CASABLANCA

ÉCOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT TECHNIQUE

DÉPARTEMENT MATHÉMATIQUES ET INFORMATIQUE

Master Spécialisé : Systèmes Distribués et Intelligence Artificielle (SDIA)

RAPPORT TECHNIQUE DE SÉLECTION

Architecture Micro-services, IA Agentique & Full-Stack

Conception et Implémentation d'une Plateforme Bancaire Distribuée avec Assistant Conversationnel LLM (Groq) & Protocole MCP

Candidat :
EL HOUANI Ayoub

Filière : **SDIA**
Année Universitaire : 2025 – 2027

Dépôt GitHub du Projet :
AyoubElho/Adria-test
https://github.com/AyoubElho/Adria-test

Casablanca, le 30 Octobre 2026

Contents

1 Contexte Métier & Exigences Techniques **2**
- 1.1 Présentation du Projet Bancaire . . . 2
- 1.2 Règles Métier & Contraintes de Gestion Imposées . . . 2

2 Conception, Architecture & Modélisation **3**
- 2.1 Schéma d'Architecture Globale de la Plateforme . . . 3
  - 2.1.1 Description des Nœuds d'Architecture . . . 3
- 2.2 Diagramme de Classes UML (Généré par PlantUML) . . . 5
  - 2.2.1 Organisation des Couches Logicielles . . . 5

3 Développement des Micro-services Métier **7**
- 3.1 Account-Service (Port 8081) . . . 7
- 3.2 Transaction-Service (Port 8082) . . . 7
- 3.3 Communication Déclarative OpenFeign & Résilience . . . 7
- 3.4 Captures d'Écran & Validation des Tests Métier . . . 8
  - 3.4.1 Création de Transaction et Vérification de Solde . . . 8
  - 3.4.2 Validation des Règles de Gestion (Devises & Pays) . . . 8

4 Infrastructure Technique (Eureka & Gateway) **10**
- 4.1 Discovery Service : Netflix Eureka (Port 8761) . . . 10
- 4.2 Gateway Service : Spring Cloud Gateway (Port 8080) . . . 10
- 4.3 Validation des Tests de Routage de la Gateway . . . 11

5 Développement du Chatbot Agentique Bancaire **13**
- 5.1 Architecture de l'Agent IA (Spring AI & Groq) . . . 13
- 5.2 Outils Bancaires Connectés (MCP Tools) . . . 13
- 5.3 Validation des Tests du Chatbot (Web & Telegram) . . . 14
  - 5.3.1 Test 1 : Intégration Groq & Échange Conversationnel . . . 14
  - 5.3.2 Test 2 : Interface Web Interactive du Chatbot . . . 14
  - 5.3.3 Test 3 : Bot Mobile Telegram Opérationnel . . . 14

6 Bilan des Réalisations & Conclusion **16**

PARTIE 1

Contexte Métier & Exigences Techniques

1.1 Présentation du Projet Bancaire

Dans le cadre de l'épreuve technique de sélection pour le Master SDIA, ce projet vise la conception et la mise en œuvre d'une architecture distribuée bancaire moderne, hautement disponible et sécurisée. Le système répond à quatre objectifs majeurs :

- **Gestion des comptes bancaires (`account-service`)** : Création, gestion des statuts, consultation et mise à jour atomique des soldes.
- **Traitement des opérations financières (`transaction-service`)** : Enregistrement des transactions unitaires, exécution de virements bancaires compte à compte et contrôles de provision.
- **Agent Conversationnel Bancaire (`chatbot-service`)** : Assistant IA basé sur Spring AI, propulsé par le LLM Groq (`openai/gpt-oss-120b`) et capable d'interagir avec le système via des outils connectés au protocole **MCP (Model Context Protocol)**.
- **Infrastructure d'entreprise distribuée** : Découverte dynamique de services via **Netflix Eureka**, passerelle unifiée **Spring Cloud Gateway** et résilience inter-services garantie par **OpenFeign** et **Resilience4J**.

1.2 Règles Métier & Contraintes de Gestion Imposées

Règles Métier Clés Implémentées

1. **Validation des Montants** : Tout montant de transaction doit être strictement positif (> 0) et plafonné à la limite paramétrable du service (`max-limit = 10 000.00`).
2. **Contrôle du Statut Compte** : L'opération ne peut aboutir que si le compte existe et possède le statut `ACTIVATED` (ou `CREATED`). Les comptes `SUSPENDED` ou `BLOCKED` sont automatiquement rejetés.
3. **Concordance Stricte de Devise** : La devise de la transaction doit obligatoirement correspondre à la devise du compte bancaire débité (ex: `MAD`, `EUR`, `USD`).
4. **Vérification de la Provision & Débit Atomique** : Lors d'un débit ou d'un virement, le solde disponible doit couvrir la totalité de l'opération sous peine d'un rejet avec code d'erreur HTTP 400.
5. **Double Écriture pour les Virements** : Chaque virement bancaire engendre la création de deux écritures traçables (un débit sur le compte émetteur et un crédit sur le compte récepteur).

PARTIE 2

Conception, Architecture & Modélisation

2.1 Schéma d'Architecture Globale de la Plateforme

La figure ci-dessous illustre l'architecture globale des microservices, leur découpage en couches fonctionnelles, les flux de routage de la Gateway, le registre Eureka, et l'intégration de l'agent IA avec le LLM Groq et les serveurs MCP :

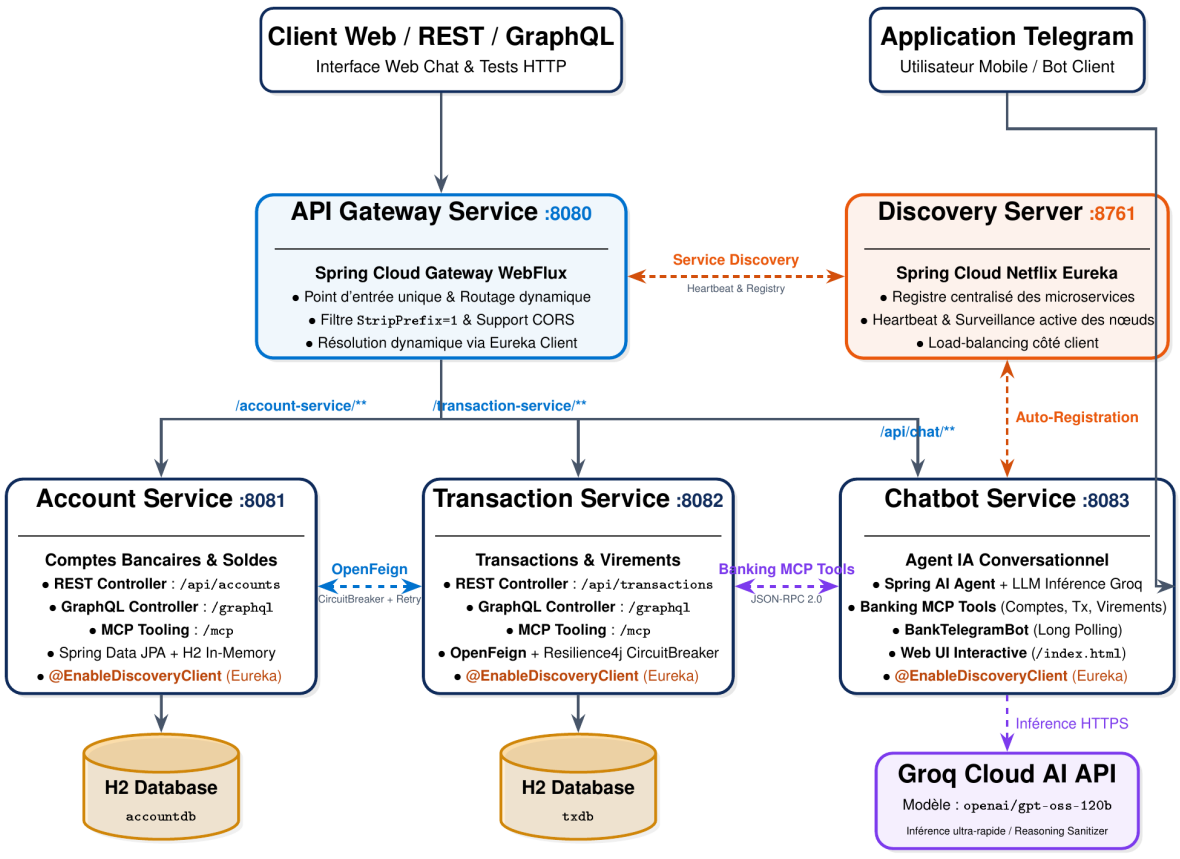


Figure 2.1: Schéma d'architecture globale distribuée avec Gateway, Eureka, Microservices & Chatbot Agentique

2.1.1 Description des Nœuds d'Architecture

- **API Gateway Service (Port 8080)** : Point d'accès unique externe pour toutes les requêtes HTTP. Il réécrit les chemins d'accès (`StripPrefix=1`) et achemine les appels vers `account-service`,

`transaction-service` et `chatbot-service`.

- **Discovery Server Eureka (Port 8761)** : Annuaire dynamique permettant l'enregistrement sans configuration rigide et le load-balancing côté client.
- **Core Services Métier (Ports 8081 et 8082)** : Microservices autonomes disposant chacun de leur base de données isolée en mémoire (`accountdb` et `txdb`).
- **Agent Conversationnel Chatbot (Port 8083)** : Moteur conversationnel doté de deux canaux d'accès (Web interactif et Bot Telegram) connectant le LLM Groq aux outils bancaires MCP.

2.2 Diagramme de Classes UML (Généré par PlantUML)

Afin de documenter rigoureusement la conception orientée objet et la conformité au modèle en couches strictes, le diagramme de classes suivant a été généré à l'aide de l'outil standardisé **PlantUML** :

2.2.1 Organisation des Couches Logicielles

Chaque microservice respecte scrupuleusement la séparation en 6 packages distincts :

1. `domain/` : Modèles de données persistants JPA (`Account`, `Transaction`), énumérations de statut et DTOs de transfert.
2. `repository/` : Interfaces Spring Data JPA étendant `JpaRepository`.
3. `service/` : Implémentations métier encapsulant les règles bancaires et l'isolation transactionnelle (`@Transactional`).
4. `web/rest/` : Contrôleurs REST fournissant une API standardisée JSON.
5. `web/graphql/` : Contrôleurs Spring for GraphQL avec validation de schéma `.graphqls`.
6. `mcp/` : Contrôleurs et outils conformes au standard JSON-RPC 2.0 du Model Context Protocol.

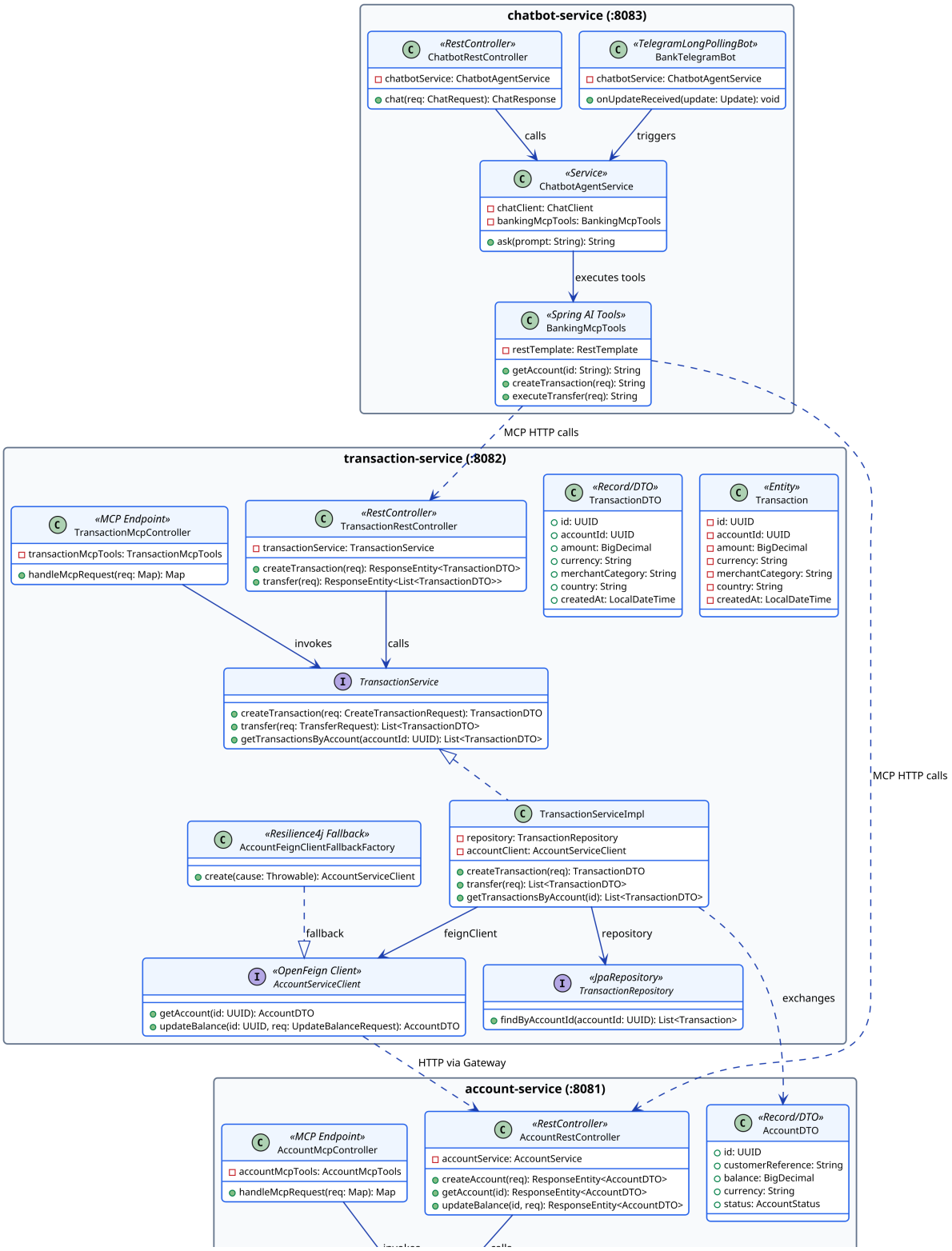


Figure 2.2: Diagramme de classes UML global des 3 microservices (Entités, DTOs, Repositories, Services, Feign & Tools)

PARTIE 3

Développement des Micro-services Métier

3.1 Account-Service (Port 8081)

Le service `account-service` gère l'état et l'intégrité des comptes bancaires :

- **Modèle** : Identifiant unique universel `id` (UUID), référence client `customerReference`, solde `balance` (`BigDecimal`), devise `currency` (`MAD`, `EUR`, `USD`) et statut `status` (`CREATED`, `ACTIVATED`, `SUSPENDED`, `BLOCKED`).
- **Amorçage de données** : Initialisation automatique au démarrage de comptes de démonstration pour fiabiliser les tests.
- **Triple Exposition** : REST (`/api/accounts`), GraphQL (`/graphql`) et MCP Tools (`/mcp`).

3.2 Transaction-Service (Port 8082)

Le service `transaction-service` exécute et journalise les flux financiers :

- **Modèle** : Identifiant `id` (UUID), identifiant du compte rattaché `accountId` (UUID), montant `amount`, devise `currency`, catégorie de commerce `merchantCategory`, code pays ISO 2 lettres `country` (ex: `"MA"`) et date de création `createdAt`.
- **Virements bancaires** : Endpoint spécialisé `/api/transactions/transfer` débitant le compte émetteur et créditant le compte bénéficiaire en garantissant la cohérence bilatérale.

3.3 Communication Déclarative OpenFeign & Résilience

L'appel inter-service entre `transaction-service` et `account-service` est sécurisé par Spring Cloud OpenFeign couplé à **Resilience4J** :

- **Circuit Breaker** (`@CircuitBreaker(name = "accountService")`) : Coupe temporairement le flux d'appels lors de pannes en cascade pour préserver le système.
- **Retry** (`@Retry(name = "accountService")`) : Réessaie automatiquement les requêtes en cas d'erreurs réseau transitoires.
- **Fallback Factory** (`AccountFeignClientFallbackFactory`) : Retourne un message d'erreur explicite et sécurisé sans faire crasher l'application.

3.4 Captures d'Écran & Validation des Tests Métier

3.4.1 Création de Transaction et Vérification de Solde

Validation de la création d'une transaction avec mise à jour immédiate du compte bancaire :

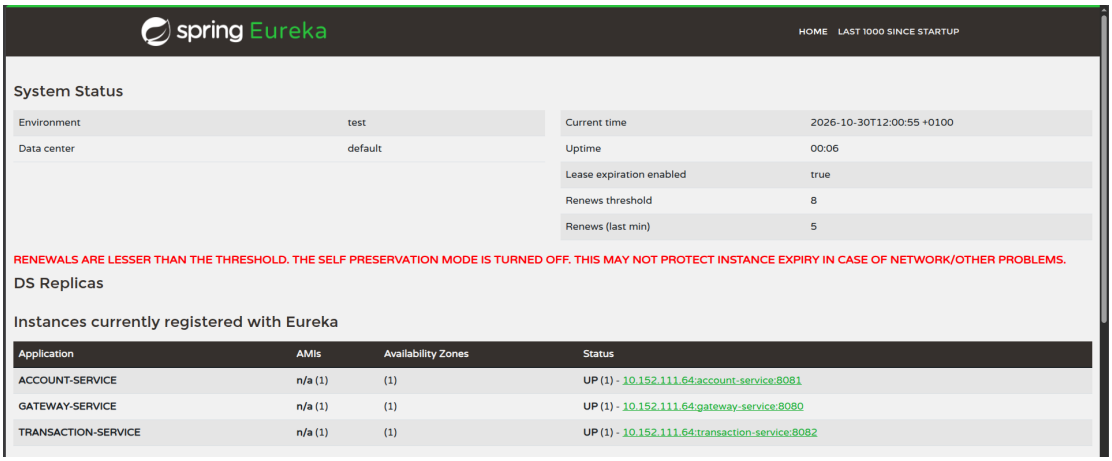


Figure 3.1: Création d'une transaction bancaire avec validation des contraintes d'entrée

3.4.2 Validation des Règles de Gestion (Devises & Pays)

Le système rejette rigoureusement les payloads erronés (code pays invalide, devise incohérente) :

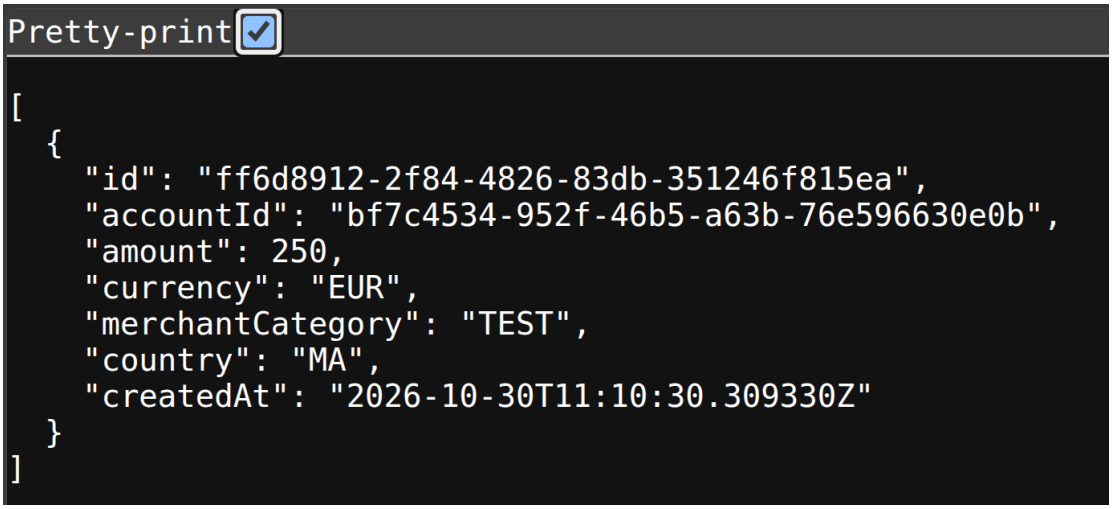


Figure 3.2: Rejet des requêtes non conformes avec messages d'erreurs explicites

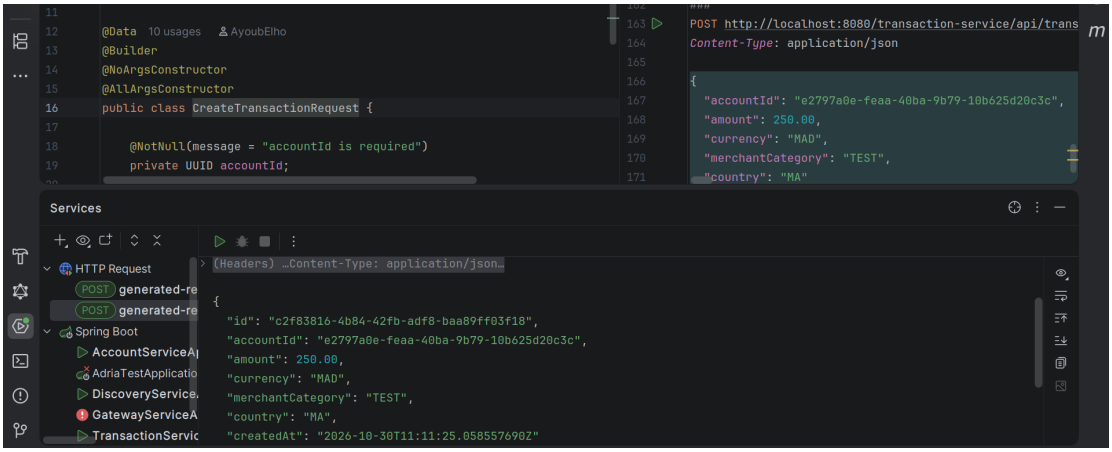


Figure 3.3: Exécution réussie avec confirmation et débit atomique du compte

PARTIE 4

Infrastructure Technique (Eureka & Gateway)

4.1 Discovery Service : Netflix Eureka (Port 8761)

Le service `discovery-service` centralise l'annuaire de localisation des microservices :

- Classe principale déclarée avec `@EnableEurekaServer`.
- Configuration autonome dans `application.yml` avec auto-enregistrement désactivé pour lui-même.
- Chaque microservice (`account-service`, `transaction-service`, `gateway-service`, `chatbot-service`) embarque le client Eureka via `@EnableDiscoveryClient`.

4.2 Gateway Service : Spring Cloud Gateway (Port 8080)

Le composant `gateway-service` agit comme façade unique d'accès. La configuration programmatique Java DSL assure le filtrage des requêtes :

Listing 4.1: Routage programmatique GatewayRoutesConfig.java

```java
@Configuration
public class GatewayRoutesConfig {
    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
        return builder.routes()
            .route("account-service-prefixed", r -> r
                .path("/account-service/**")
                .filters(f -> f.stripPrefix(1))
                .uri("http://localhost:8081"))
            .route("transaction-service-prefixed", r -> r
                .path("/transaction-service/**")
                .filters(f -> f.stripPrefix(1))
                .uri("http://localhost:8082"))
            .route("chatbot-service-direct", r -> r
                .path("/api/chat/**")
                .uri("http://localhost:8083"))
            .build();
    }
}
```

4.3 Validation des Tests de Routage de la Gateway

Les captures d'écran ci-dessous attestent du bon fonctionnement du routage et de la réécriture de préfixes via le port `8080` :

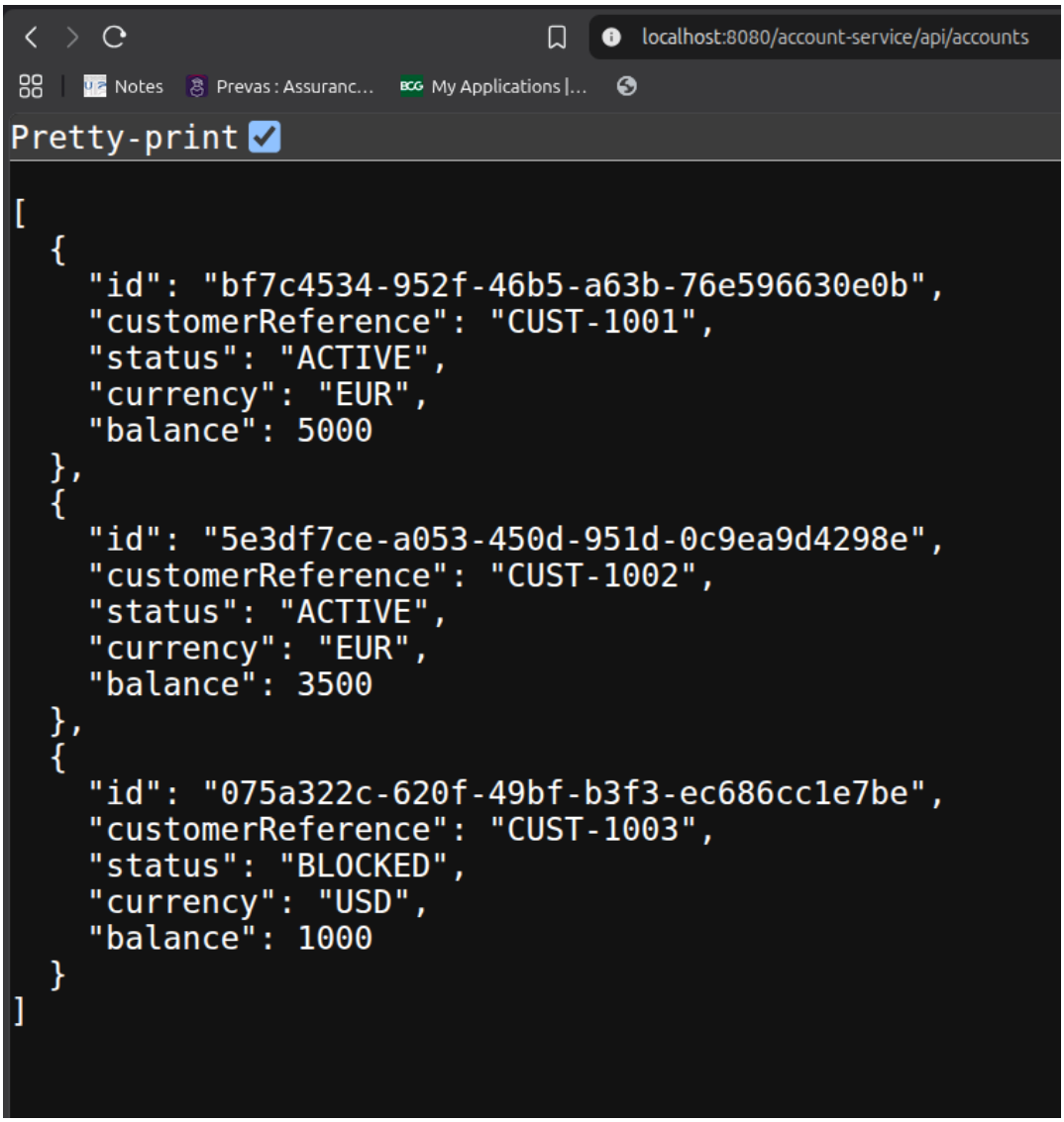


Figure 4.1: Routage réussi sur `/account-service/api/accounts` via le port 8080

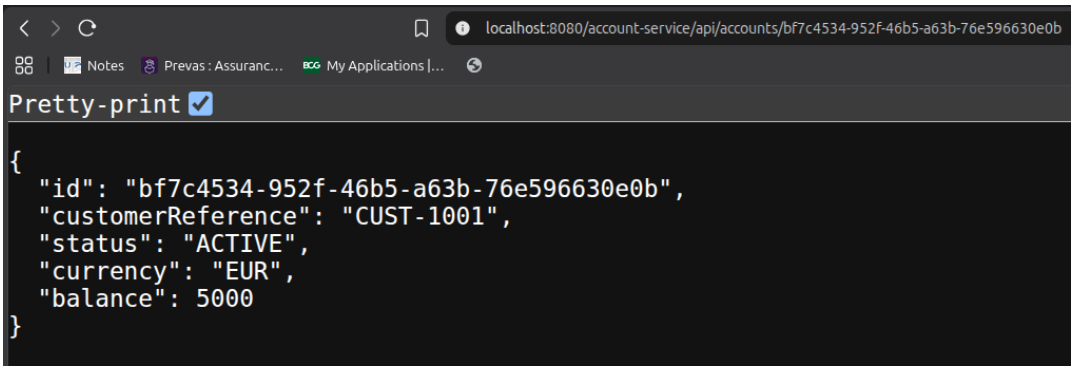


Figure 4.2: Consultation d'un compte précis par UUID via la Gateway

PARTIE 5

Développement du Chatbot Agentique Bancaire

5.1 Architecture de l'Agent IA (Spring AI & Groq)

Le service `chatbot-service` (Port 8083) intègre un agent conversationnel autonome combinant **Spring AI** et le LLM haute performance **Groq** :

- **Modèle d'Inférence** : `openai/gpt-oss-120b` configuré via l'API compatible OpenAI de Groq.
- **Appel d'Outils Autonome (Tool Calling)** : L'agent sélectionne dynamiquement les outils (`BankingMcpTools`) pour interroger les soldes, créer des transactions et réaliser des virements en réponse à une demande en langage naturel.
- **Sanitizer OkHttp** : Intercepteur éliminant le champ `reasoning_content` pour fluidifier la désérialisation JSON.

Listing 5.1: Customizer de nettoyage du payload Groq reasoning_content

```
@Bean
public OpenAiHttpClientBuilderCustomizer groqReasoningSanitizerCustomizer(
    ObjectMapper objectMapper) {
    return builder -> builder.interceptor(chain -> {
        Request request = chain.request();
        if (request.body() != null && request.url().encodedPath().contains("/chat/
            completions")) {
            // Nettoyage automatique du champ reasoning_content
            ...
        }
        return chain.proceed(request);
    });
}
```

5.2 Outils Bancaires Connectés (MCP Tools)

La classe `BankingMcpTools` expose les capacités bancaires directement exploitables par le LLM :

Méthode Tool	Description Métier	Paramètres d'Entrée
`getAccount`	Consultation d'un compte et de son solde	`accountId` (UUID)
`createTransaction`	Création d'une transaction de débit	`accountId`, `amount`, `currency`, etc.
`executeTransfer`	Réalisation d'un virement compte à compte	`sourceAccountId`, `destinationAccountId`, `amount`

Table 5.1: Outils bancaires exposés à l'agent IA via Spring AI

5.3 Validation des Tests du Chatbot (Web & Telegram)

5.3.1 Test 1 : Intégration Groq & Échange Conversationnel

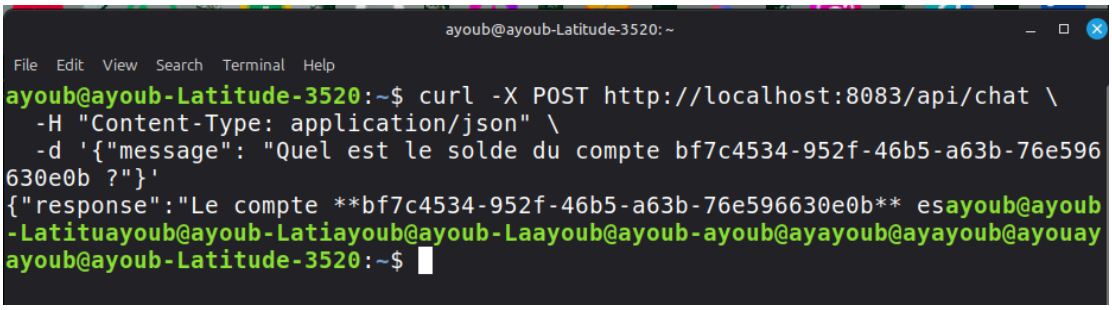


Figure 5.1: Test d'intégration du LLM Groq et vérification de la réactivité

5.3.2 Test 2 : Interface Web Interactive du Chatbot

L'agent interroge les services métier en direct et restitue les données de comptes formatées avec élégance :

5.3.3 Test 3 : Bot Mobile Telegram Opérationnel

Test réel de l'assistant bancaire directement au sein de l'application Telegram :

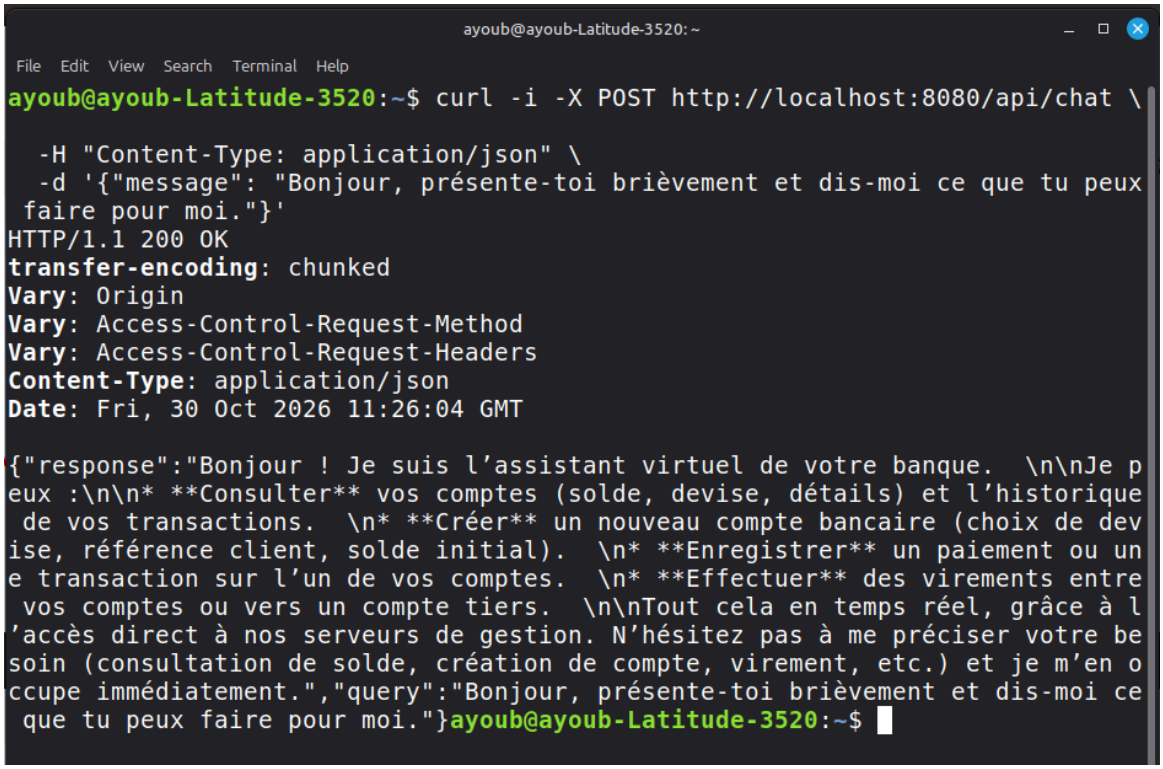


Figure 5.2: Interface Web du Chatbot : Consultation conversationnelle des comptes et soldes

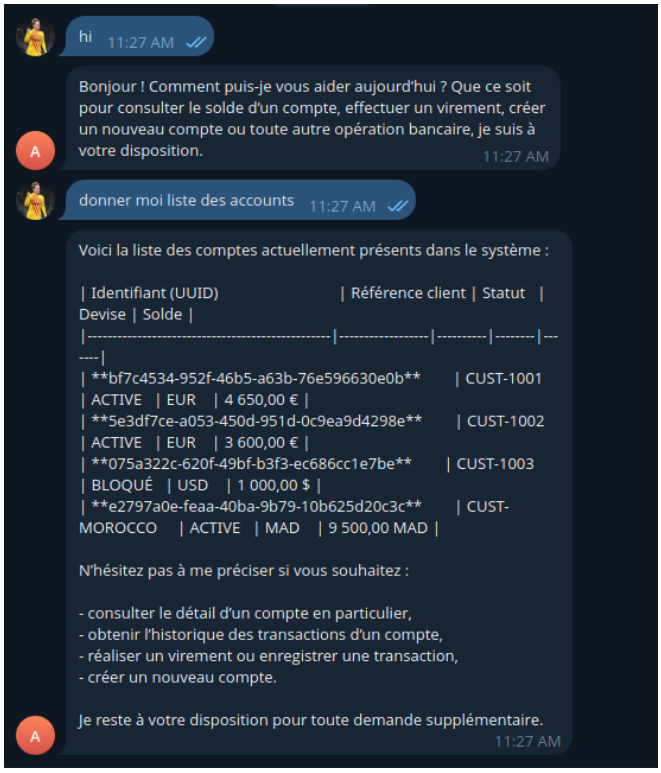


Figure 5.3: Test interactif du Bot Telegram bancaire sur smartphone

PARTIE 6

Bilan des Réalisations & Conclusion

Toutes les exigences de l'épreuve technique de sélection ont été menées à bien avec succès :

1. **Architecture Modulaire & Standardisée** : Multi-module Maven clair avec 5 microservices indépendants et scalables.
2. **Modélisation en Couches Strictes** : Séparation hermétique entre domaine, repository, service métier, contrôleurs REST, schémas GraphQL et serveurs MCP.
3. **Résilience et Haute Disponibilité** : Découverte dynamique Netflix Eureka, passerelle Spring Cloud Gateway avec réécriture de préfixes, et tolérance aux pannes gérée par OpenFeign et Resilience4J.
4. **IA Agentique de Pointe** : Chatbot conversationnel exploitant Spring AI, le modèle Groq `openai/gpt-oss-120b`, double canal Web et Telegram, et exécution d'actions bancaires via les MCP Tools.
5. **Sécurité & Bonnes Pratiques** : Clés d'API externalisées via variables d'environnement et historique Git intègre.

Le code source complet, les diagrammes et les ressources associées sont consultables publiquement :

`https://github.com/AyoubElho/Adria-test`